

Andres Arroyo
Shane Friedenber
Computer Security

Final Project Report

Email-based attacks are a pretty common threat, especially if you can't tell between what is a real email or a fake one, with the majority of phishing attempts relying on spoofing, which is where an attacker forges the "From" address to impersonate a trusted sender. Since the original email protocols lacked any form of authentication, the attackers creating messages that appear legitimate happen all the time, which tricks users into clicking malicious links or sharing sensitive information. To counter this risk, modern email systems rely on authentication frameworks such as SPF (Sender Policy Framework) and DKIM (DomainKeys Identified Mail). These authentication frameworks verify the sender's authority and protect the integrity of the message, making spoofing significantly more difficult for attackers. The goal of our project was to demonstrate both sides of this: how easy it is to spoof an email, and how SPF and DKIM serve as defenses that differentiate malicious emails from legitimate ones. Our project took the form of a local, controlled demonstration that allowed us to safely explore spoofing, authentication, and header analysis without sending any real emails out onto external networks.

At the start, we defined several goals based on our project proposal. We aimed to show that spoofed messages can be delivered when no authentication is present, and to implement both SPF and DKIM for a test domain so that legitimate messages could be validated. An important part of our objectives involved capturing and analyzing

headers, because this is where SPF and DKIM results appear. We evaluated our system based on how reliable it was for spoofed messages to be accepted, how consistently authenticated messages passed validation, and whether header inspection clearly revealed differences between fake and trusted senders. The relevance of this project comes from the fact that emails continue to be the number one reason for breaches, and understanding both vulnerabilities and defenses is valuable for anyone entering cybersecurity because it is so common.

Our system architecture is a simple model but is effective. We created two simulated senders. One sender representing Bank of America (the legitimate sender with SPF and DKIM configured), and another representing DU Students (a spoofing domain with no authentication). Both senders transmitted their messages to a local SMTP server built using aiosmtpd, which saved all incoming emails as raw .eml files. This mailbox served as the data source for SPF checks, DKIM verification, and header analysis. SPF was implemented as a lightweight simulation that compared the sender's IP address to a domain's authorized list. DKIM used real RSA keypairs, where messages from the legitimate sender were signed using the private key and later verified with the matching public key. By sending both legitimate and spoofed messages through the same pipeline, we created a controlled environment that made authentication outcomes easy to observe and compare.

The implementation was done entirely in Python using VS Code. The dependencies included aiosmtpd for the local server, dkimpy for signatures and verification, and cryptography for RSA key generation. Our SMTP server (smtp_server.py) listened on port 1025 and wrote each received message to disk as a

timestamped .eml file. The SPF simulation script (spf_check.py) compared each sender's domain and IP address to determine whether sending was authorized. DKIM keys were created using our key generator (generate_client_keys.py), which produced a private and public key for each domain. The multi-client demo (multi_client_demo.py) handled the process of constructing, signing, sending, and verifying emails. For header analysis, we used a script (header_analysis.py) that extracted "From," "To," "Subject," and "DKIM-Signature" fields for comparison across messages.

The two senders behaved very differently and allowed us to understand the contrast between spoofed and authenticated messages. The Bank of America sender used both SPF and DKIM. Its SPF policy authorized the local machine's IP address, and DKIM signing added a cryptographic signature to its "From," "To," and "Subject" headers. A captured signed email shows this signature in detail (msg_20251109T215133991023Z). By contrast, the DU Students sender sent an unsigned message with a forged "From" field, identical to how a real attacker might construct a phishing attempt. An example spoofed email captured by our server shows simple plain-text content and no DKIM-Signature header (msg_20251109T215134821272Z).

The results of our project were clear and aligned with what we expected from the beginning. The legitimate sender passed SPF because its IP address matched the one listed in the SPF policy, while the spoofed one failed immediately because it attempted to send from an unauthorized address. DKIM validation also succeeded for the legitimate message, both in memory and after being saved to a .eml file. This confirmed that the cryptographic signature remained unbothered throughout the whole process.

The spoofed message, as expected, lacked any signature and failed DKIM. The combination of SPF and DKIM allowed us to classify the legitimate message as trusted and the spoofed one as suspicious or rejected. These distinctions were easy to confirm through the header inspection tool.

One of the more interesting discoveries we made was how sensitive DKIM is to formatting. Even minor changes to line endings or whitespace can break DKIM verification. This strictness is intentional, and any modification to the signed portion of a message should invalidate the signature to prevent tampering. However, it also means that implementing DKIM requires careful attention to canonicalization rules. Debugging these issues improved our understanding of low level message integrity and the byte-exact nature of signatures.

Overall, this project gave us a deep appreciation for how spoofing and authentication work behind the scenes. We learned firsthand how simple it is to forge an email and how essential SPF and DKIM are in preventing such attacks. Working with real cryptographic signatures, RSA keypairs, and message parsing provided actual experience that is more fulfilling than something like learning information for an exam. We also gained a better understanding of layered defenses, because SPF alone cannot prevent spoofing, and DKIM alone cannot prevent unauthorized senders, but together they greatly reduce the likelihood of a successful phishing attempt.

In conclusion, our implementation successfully demonstrated both the vulnerability of unauthenticated email and the effectiveness of modern protections. By building a functional authentication pipeline, signing and verifying messages, modeling spoofed behavior, and analyzing the resulting headers, we met all of our project goals.

We showed a clear difference between fake and verified emails and gained valuable experience with SPF, DKIM, and foundational security practices. This project not only deepened our understanding of email authentication but also reinforced the importance of layered defenses in modern cybersecurity.